Smart Lender – Loan Approval Prediction

Project Description:

Smart Lender – Loan Approval Prediction is a Machine Learning-based web application that predicts whether a loan application is likely to be approved or rejected based on applicant information. The system helps financial institutions make faster and more consistent loan decisions by analyzing customer details such as gender, marital status, education, income, loan amount, credit history, and property area.

The application is developed using Python, Flask, HTML, CSS, and JavaScript, while the prediction model is built using the XGBoost Machine Learning algorithm. The trained model is saved as a .pkl file and integrated into the Flask backend to provide real-time predictions through an easy-to-use web interface.

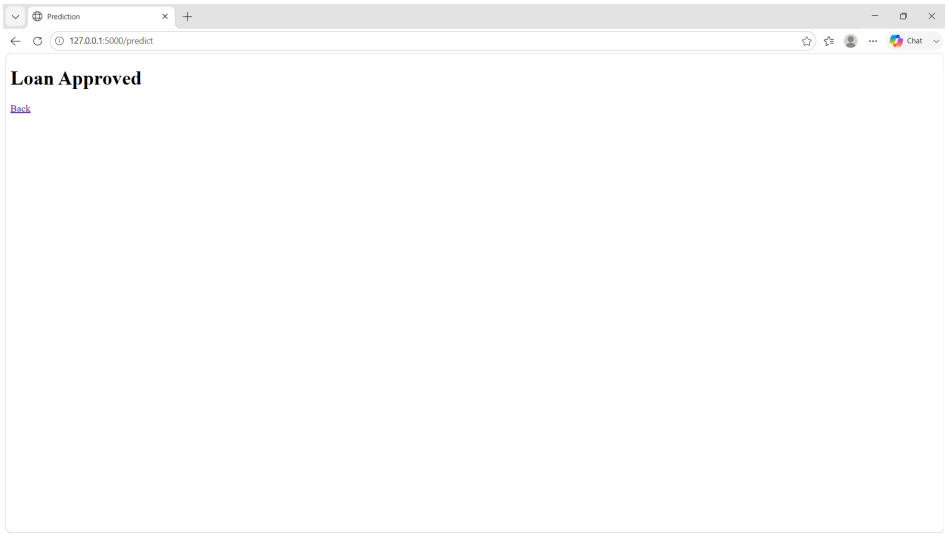
The project is deployed on Render, allowing users to access the application online. The user enters loan details through a web form, and the application instantly predicts whether the loan is approved or rejected.

Scenarios

Scenario 1 – Loan Approved

A salaried applicant with a good credit history, stable income, and moderate loan amount submits the application. The system processes the data through the trained XGBoost model and predicts:

Loan Approved

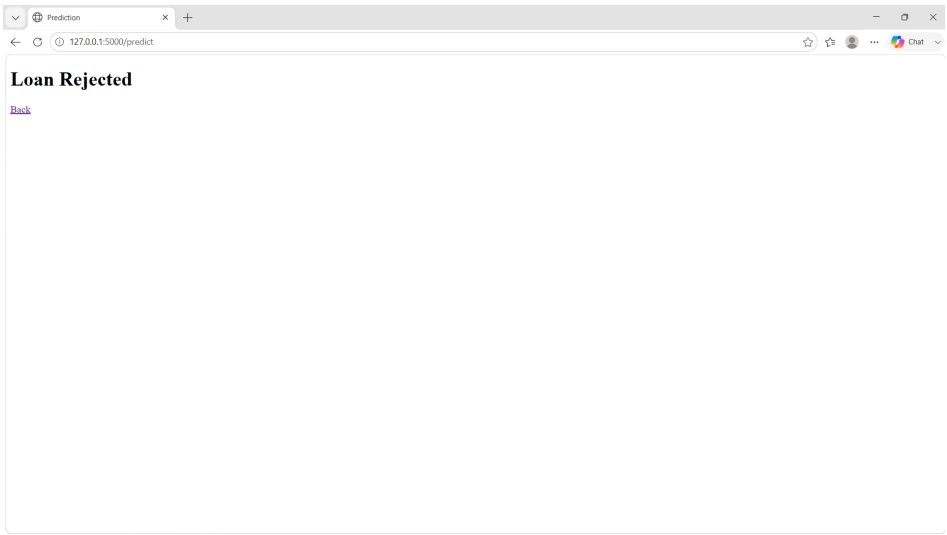


Loan Approved Result Screenshot

Scenario 2 – Loan Rejected

An applicant with low income, poor credit history, or high loan amount applies for a loan. The machine learning model evaluates the input and predicts:

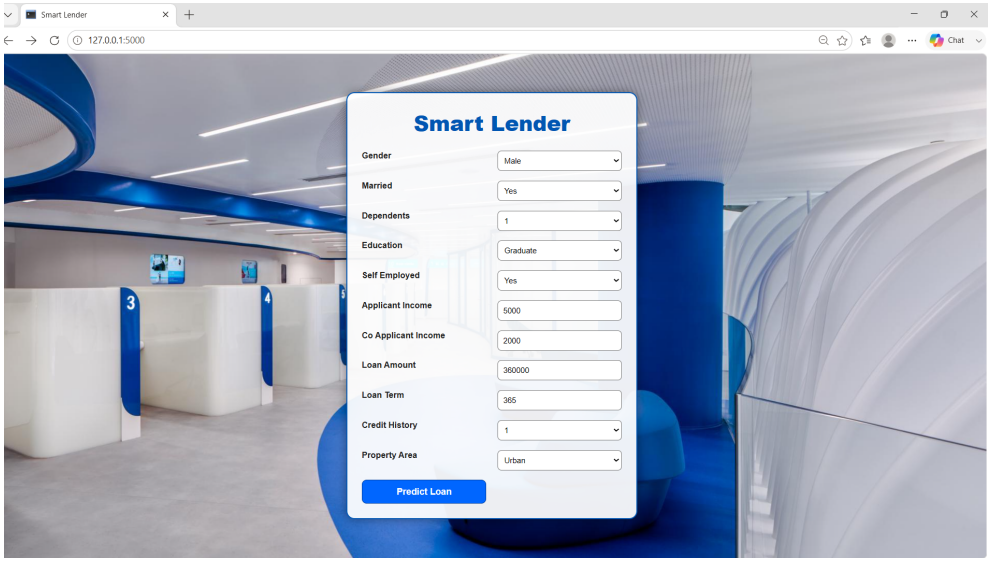
Loan Rejected



Loan Rejected Result Screenshot

Scenario 3 – New Loan Application

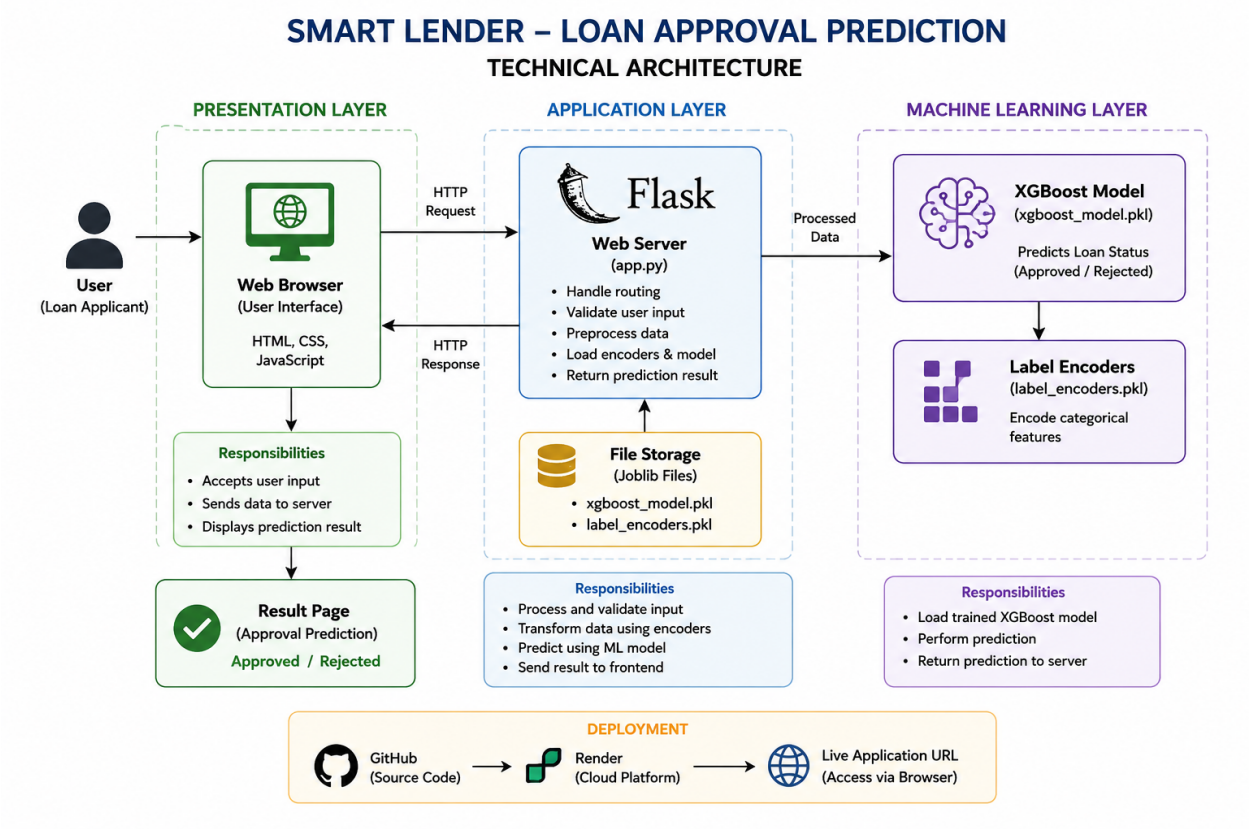
A user opens the Smart Lender website, enters applicant information such as income, loan amount, education, marital status, and credit history, then clicks Predict Loan. The application processes the request and immediately displays the prediction.



Home Page Screenshot

Technical Architecture

The Smart Lender application follows a three-layer architecture consisting of the Presentation Layer (Web Browser interface built with HTML, CSS, and JavaScript), the Application Layer (Flask backend that validates input, loads the trained model and encoders, and returns the prediction result), and the Machine Learning Layer (the XGBoost model and label encoders that generate the loan approval prediction).



Technical Architecture Diagram

System Workflow

User Input → Flask Backend → Preprocessing → XGBoost Model → Prediction → Display Result.

Dataset Information

Loan Prediction Dataset from Kaggle with 614 records, 13 input features and Loan_Status target.

Feature Description

Features include Gender, Married, Dependents, Education, Self_Employed, ApplicantIncome, CoapplicantIncome, LoanAmount, Loan_Amount_Term, Credit_History, Property_Area, Loan_Status.

Data Preprocessing

Handled missing values, encoded categorical variables using LabelEncoder, selected features and split data into 80% training and 20% testing.

Model Selection

XGBoost selected for high accuracy, speed and handling of tabular data.

Model Evaluation

Include Accuracy, Precision, Recall, F1-Score and Confusion Matrix using your actual results.

Technologies Used

Python, Flask, HTML, CSS, JavaScript, Pandas, NumPy, Scikit-learn, XGBoost, Joblib, GitHub and Render.

Pre-requisites

- Python 3.10 or above
- Flask Framework
- HTML
- CSS
- JavaScript
- XGBoost
- Pandas
- NumPy
- Scikit-learn
- Joblib
- Git & GitHub
- Render (Deployment)

Project Workflow

Milestone 1 – Data Collection & Model Development

Activity 1.1: Collect the Loan Prediction dataset.

Activity 1.2: Clean the dataset and preprocess missing values.

Activity 1.3: Encode categorical values using Label Encoder.

Activity 1.4: Train the XGBoost Machine Learning model.

Milestone 2 – Backend Development

Activity 2.1: Develop the Flask backend using app.py.

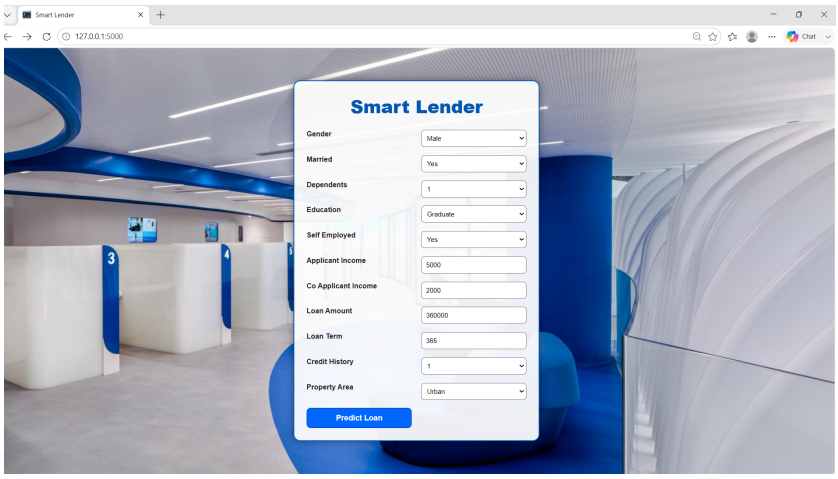
Activity 2.2: Load the trained XGBoost model.

Activity 2.3: Accept user input from the web application.

Activity 2.4: Predict loan approval.

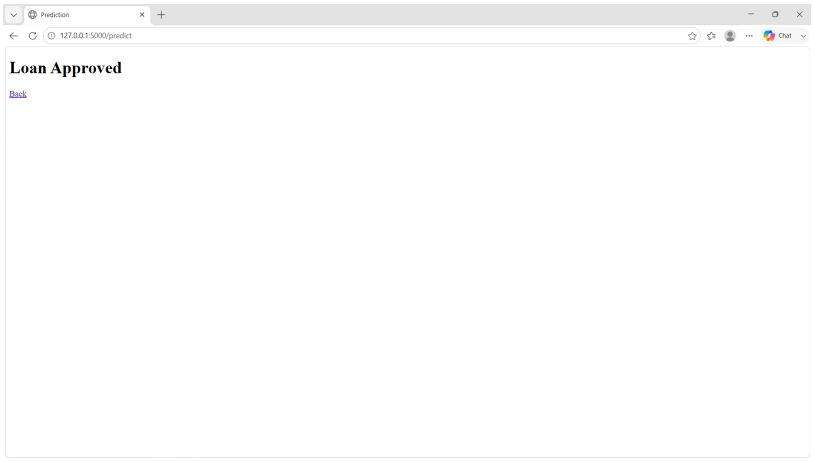
Milestone 3 – Frontend Development

Activity 3.1: Develop the Loan Application Form.

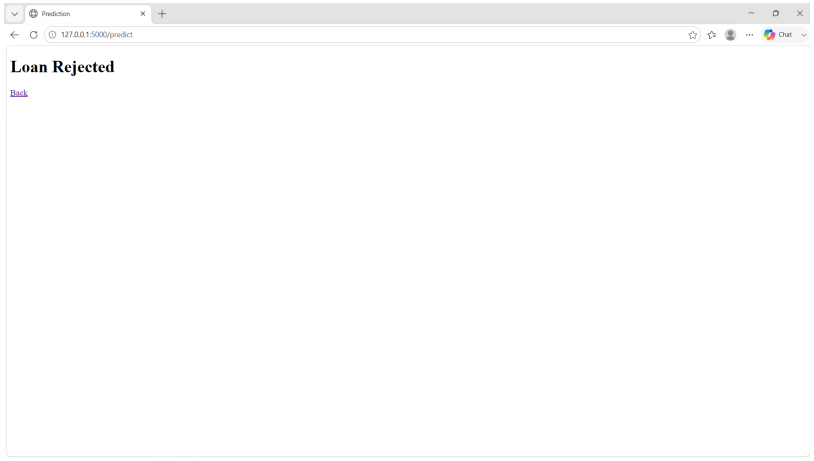


Home Page Screenshot

Activity 3.2: Display prediction results.



Loan Approved Screenshot

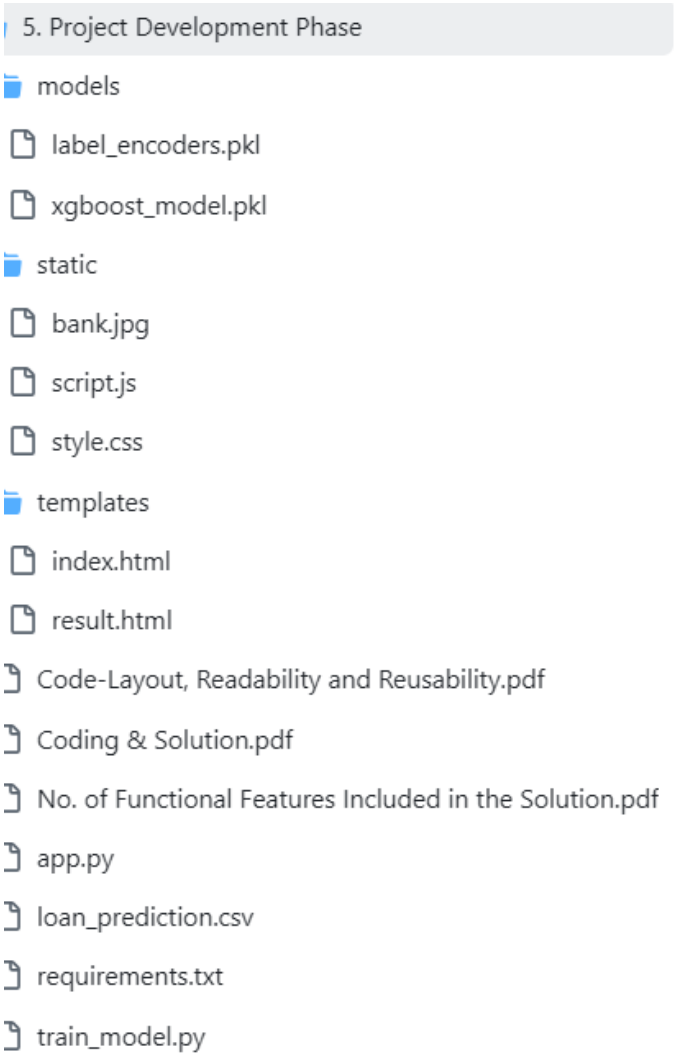


Loan Rejected Screenshot

Milestone 4 – Deployment

Activity 4.1: Preparing the Application for Local Deployment

Set Up Application Structure: Create the project directory structure including folders for models, static files, and templates, and the main application files (app.py, train_model.py, loan_prediction.csv, requirements.txt).



Project Folder Structure

Set Up a Virtual Environment: Create and activate a virtual environment, then install all required dependencies from requirements.txt:

```
D:\projects>python -m venv myenv
D:\projects>"d:\projects\venv\Scripts\activate"
(myenv) D:\projects>pip install -r requirements.txt
```

Activity 4.2: Local Testing and Verification

Start the Flask Development Server: Run the application locally using:

```
(myenv) D:\projects>python app.py
```

Once running, open a browser and navigate to “http://127.0.0.1:5050” to access the app.

- Once running, open a browser and navigate to "http://127.0.0.1:5050" to access the app.

Flask Development Server Running

Interact with the Smart Lender form test different applicant scenarios (approved and rejected cases) to verify the predictions are correctly generated, processed, and displayed.

Activity 4.3: Deploy on Render

Deploy the Smart Lender application using Render.

Live URL: https://smart-lender-ml-project.onrender.com/

Conclusion

The Smart Lender – Loan Approval Prediction project successfully demonstrates the use of Machine Learning in banking and financial services. Using the XGBoost algorithm, the system accurately predicts whether a loan application is likely to be approved or rejected based on applicant details.

The application provides a user-friendly web interface developed with Flask, HTML, CSS, and JavaScript, allowing users to enter loan information and receive instant predictions. Deployment on Render makes the application easily accessible online.

Future improvements include integrating user authentication, storing loan application history in a database, adding visual analytics dashboards, supporting multiple machine learning models for comparison, and enhancing prediction accuracy with larger datasets. These improvements can make Smart Lender a more comprehensive decision-support system for financial institutions.

Challenges Faced

Missing values, encoding categorical data, model integration, deployment and improving prediction accuracy.

Future Enhancements

Authentication, database integration, dashboards, explainable AI, application history and improved models.

References

Kaggle Loan Prediction Dataset, Flask, XGBoost, Scikit-learn, Pandas and Render documentation.